

16주차: DeepCache: Accelerating Diffusion Models for Free

0. Abstract

디퓨전 모델은 이미지 합성 분야에서 뛰어난 성능을 보이지만 순차적인 디노이징 과정과 큰 모델 크기로 인해 계산 비용이 많이 듭니다. 기존 압축 방법들은 재학습이 필요해 비용과 실행 가능성에 제약이 있음

모델 구조 관점에서 디퓨전 모델을 가속화하는 새로운 학습 없는 방법 제안. 디퓨전 모델의 순차적 디노이징 단계에서 나타나는 시간적 중복성을 활용. 인접한 디노이징 단계 간의 특징을 캐시하고 재사용. U-Net의 특성을 활용해 고수준 특징은 재사용하고 저수준 특징은 저비용으로 업데이트

속도 향상됨. 기존 가지치기나 증류 방법보다 우수한 성능. DDIM이나 PLMS와 같은 현재 샘플링 기법들과 호환 가능. 동일한 처리량에서 비슷하거나 더 나은 결과 달성

1. Introduction

배경과 문제점: 디퓨전 모델은 이미지, 텍스트, 오디오, 비디오 생성 등 다양한 분야에서 큰 발전을 이룸. 이미지 편집, 초해상도, 이미지 변환, 텍스트-이미지 생성, 텍스트-3D 생성 등에 활용. 하지만 느린 추론 속도가 광범위한 활용의 주요 장애물. 단계적 디노이징 과정이 필요해 병렬 처리가 제한됨

기존 해결 방안과 한계: 샘플링 단계 수를 줄이는 방법. 모델 가지치기, 증류, 양자화를 통한 단계별 추론 비용 감소. 네트워크 구조 재설계나 주파수 사전 정보 활용. 하지만 대규모 데이터셋으로 재학습이 필요하거나 비용이 많이 듭니다

DeepCache: 추가 학습 없이 각 디노이징 단계의 계산 비용을 줄이는 방법 제안함. 연속된 단계 간 고수준 특징의 시간적 일관성을 활용. U-Net의 구조적 특성을 이용해 고수준 특징은 캐시하고 저수준 특징만 업데이트.

런타임에서 동적으로 디퓨전 모델을 압축하는 DeepCache 알고리즘 제안. 고수준 특징의 시간적 일관성을 활용해 중복 계산 감소. 긴 캐싱 간격을 위한 1:N 전략 도입. 다양한 데이터셋과 모델에서 검증됨. 재학습이 필요한 기존 방법들보다 우수한 성능 달성

3. Methodology

3.1 Preliminary

디퓨전 모델의 과정:

Forward Process: 실제 데이터 분포에서 샘플링한 데이터에 점진적으로 가우시안 노이즈 추가

Reverse Process: 랜덤 노이즈에서 시작해 목표 분포로 디노이징.

U-Net은 다운샘플링 블록과 업샘플링 블록의 쌍으로 구성. 스킵 연결로 블록 간 연결

메인 브랜치: 고수준 특징 처리

스킵 브랜치: 저수준 특징 처리

저수준, 고수준 특징 효과적으로 결합. 생체의학 이미지 분할에서 시작해 강력한 성능 입증. 스킵 연결을 통해 풍부한 저수준 정보 적절 전달

3.2 Feature Redundancy in Sequential Denoising

문제점: 디퓨전 모델의 순차적 디노이징 과정이 추론 속도의 주요 병목. 기존 방법들은 주로 특정 단계를 건너뛰는 전략 사용

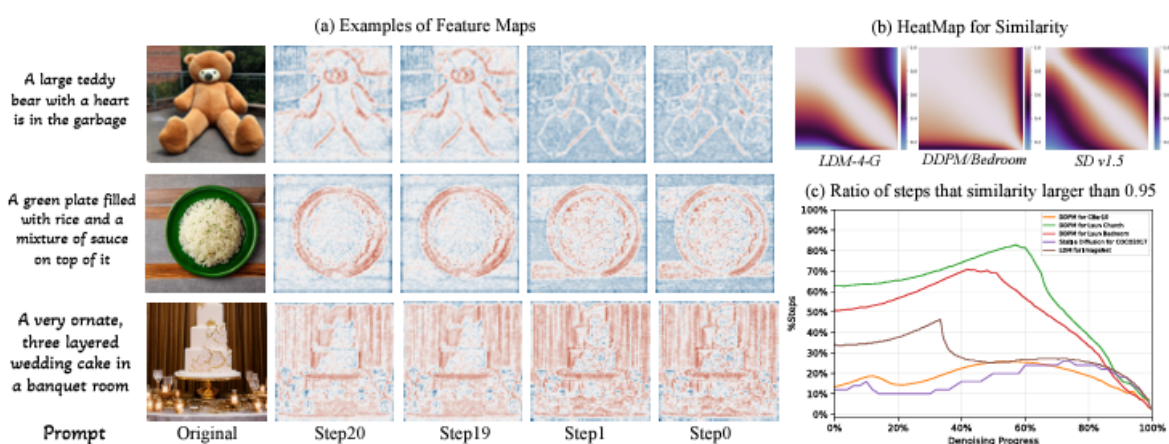
관찰 결과: 디노이징 과정의 인접 단계들이 고수준 특징에서 시간적 유사성을 보임. 연속된 단계 간의 변화가 일반적으로 미미. 각 타임스텝에서 최소 10%의 인접 타임스텝이 현재 단계와 높은 유사도

모델별 관찰

Stable Diffusion, LDM, DDPM 등 다양한 모델에서 이 현상 발견

LSUN-Churches와 LSUN-Bedroom 데이터셋의 DDPM의 일부 타임스텝은 다른 단계들의 80%와 높은 유사도 보임

최적화 가능성: 이전 단계와 매우 유사한 특징을 다시 계산하는 중복 연산 존재. 유사한 특징 맵 재생성에 많은 계산 자원 할당은 비효율적. 이러한 특성을 활용해 디노이징 과정 가속화 가능성 제시



3.3 Deep Cache For Diffusion Models

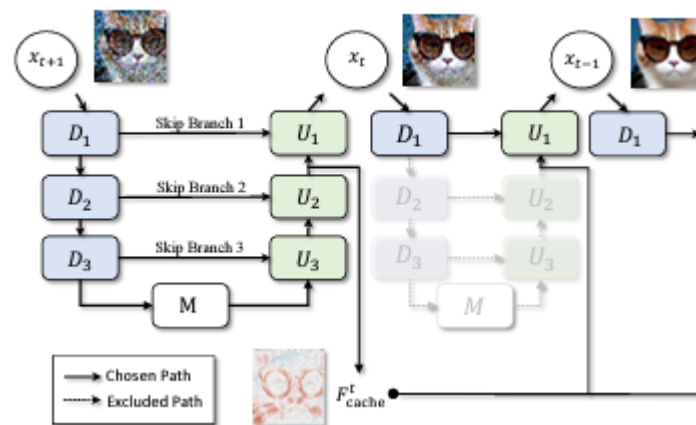
DeepCache: 컴퓨터 시스템의 캐싱 메커니즘에서 영감을 받음. 시간에 따라 최소한의 변화만 보이는 요소들을 저장.

U-Net의 스킵 연결 활용해 이중 경로 이점 활용:

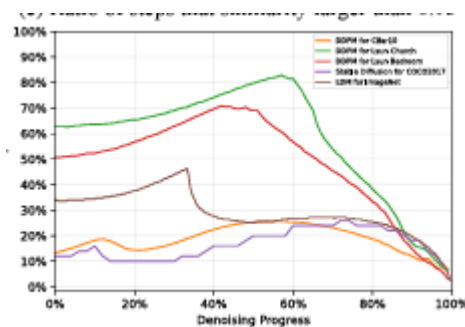
- 메인 브랜치: 전체 네트워크 통과
- 스킵 브랜치: 얇은 층만 통과

캐싱 과정: 타임스텝 t 와 $t-1$ 의 연속된 두 단계에서 작동. 이전 단계의 고수준 특징을 캐시하고 다음 단계에서 재사용. 전체 네트워크 대신 부분적 추론 수행. 스킵 브랜치에서 필요한 계산만 수행하고 메인 브랜치는 캐시된 특징 활용

1:N 추론 확장: 하나의 전체 추론 후 여러 단계의 부분 추론 가능. 캐시된 특징을 $N-1$ 개의 연속된 단계에서 재사용. 전체 T 단계에서 $k = \lceil T/N \rceil$ 횟수만큼 캐시 업데이트



비균일 1:N 추론: 큰 N 값에서 고수준 특징의 불변성이 약해지는 문제 해결. 특징 유사도가 낮은 단계에서 더 많은 샘플링 수행. 중심 타임스텝을 기준으로 이차함수적으로 인덱스 빈도 변화. 특정 타임스텝 중심의 다른 시퀀스도 유사한 품질 개선 가능



4. Experiment

4.1 Experimental Settings

DDPM: 100 단계 DDIM 샘플링 사용

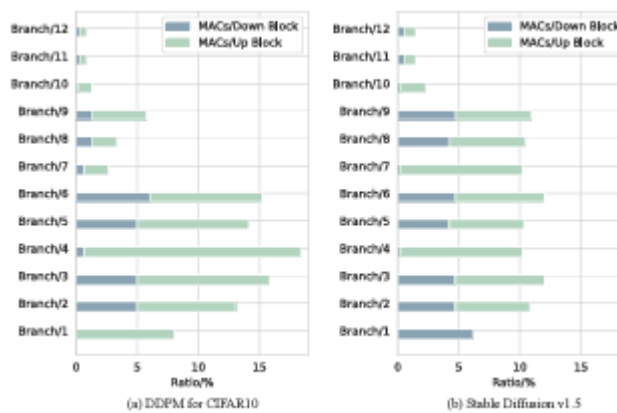
LDM: 250 스텝 사용

Stable Diffusion 50 step PLMS 사용

평가 데이터셋: CIFAR10, LSUN, ImageNet

평가지표: FID, sFID, IS, Precision and Recall, CLIP Score

4.2 Complexity Analysis



- MACs 감소: 계산 복잡도를 낮춤
- Throughput과 속도 개선: 계산 복잡도 감소로 인해 초당 처리량 증가하며 추론 속도 개선
- U-Net의 스킵 연결 활용: DeepCache는 U-Net의 스킵 연결을 기반으로 레이어를 분할해 계산량을 분산시키며 효율성을 극대화. 계산량을 고르게 분배하는 구조는 추론 속도를 높이는 데 효과적
- 속도와 품질의 트레이드오프: 스킵 브랜치의 개수와 분배 방식에 따라 속도와 품질 사이의 균형을 조절. 속도를 우선시하면서도 품질 저하를 최소화하는 최적의 설정을 확인했습니다.

4.3 Comparison with Compression

ImageNet 256 × 256									
Method	MACs ↓	Throughput ↑	Speed ↑	Retrain	FID ↓	sFID ↓	IS ↑	Precision ↑	Recall ↑
IDDPM [42]	1416.3G	-	-	✗	12.26	5.42	-	70.0	62.0
ADM-G [9]	1186.4G	-	-	✗	4.59	5.25	186.70	82.0	52.0
LDM-4 [47]	99.82G	0.178	1×	✗	3.60	-	247.67	87.0	48.0
LDM-4*	99.82G	0.178	1×	✗	3.37	5.14	204.56	82.71	53.86
Spectral DPM [66]	9.9G	-	-	✓	10.60	-	-	-	-
Diff-Pruning [10]*	52.71G	0.269	1.51×	✓	9.27 _(9.16)	10.59	214.42 _(201.81)	87.87	30.87
Uniform - N=2	52.12G	0.334	1.88×	✗	3.39	5.11	204.09	82.75	54.07
Uniform - N=3	36.48G	0.471	2.65×	✗	3.44	5.11	202.79	82.65	53.81
Uniform - N=5	23.50G	0.733	4.12×	✗	3.59	5.16	200.45	82.36	53.31
Uniform - N=10	13.97G	1.239	6.96×	✗	4.41	5.57	191.11	81.26	51.53
Uniform - N=20	9.39G	1.876	10.54×	✗	8.23	8.08	161.83	75.31	50.57
NonUniform - N=10	13.97G	1.239	6.96×	✗	4.27	5.42	193.11	81.75	51.84
NonUniform - N=20	9.39G	1.876	10.54×	✗	7.11	7.34	167.85	77.44	50.08

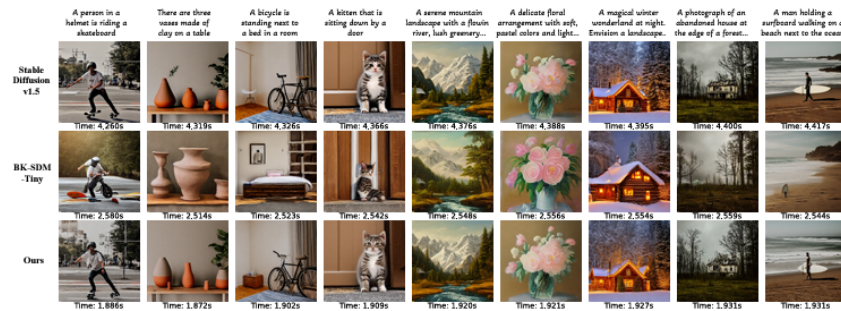
ImageNet 에서 실험했을때 1:N에서 N이 클 수록 많이 향상됨

CIFAR-10 32 × 32					
Method	MACs ↓	Throughput ↑	Speed ↑	Steps ↓	FID ↓
DDPM	6.1G	9.79	1×	-	4.19
DDPM*	6.1G	9.79	1×	-	4.16
Diff-Pruning	3.4G	13.45	1.37×	100k	5.29
Ours - N=2	4.15G	13.73	1.40×	0	4.35
Ours - N=3	3.54G	15.74	1.61×	0	4.70
Ours - N=5	3.01G	18.11	1.85×	0	5.73
Ours - N=10	2.63G	20.26	2.07×	0	9.74
LSUN-Bedroom 256 × 256					
Method	MACs ↓	Throughput ↑	Speed ↑	Steps ↓	FID ↓
DDPM	248.7G	0.21	1×	-	6.62
DDPM*	248.7G	0.21	1×	-	6.70
Diff-Pruning	138.8G	0.31	1.48×	200k	18.60
Ours - N=2	190.8G	0.27	1.29×	0	6.69
Ours - N=3	172.3G	0.30	1.43×	0	7.20
Ours - N=5	156.0G	0.31	1.48×	0	9.49
LSUN-Churches 256 × 256					
Method	MACs ↓	Throughput ↑	Speed ↑	Steps ↓	FID ↓
DDPM	248.7G	0.21	1×	-	10.58
DDPM*	248.7G	0.21	1×	-	10.87
Diff-Pruning	138.8G	0.31	1.48×	500k	13.90
Ours - N=2	190.8G	0.27	1.29×	0	11.31
Ours - N=3	172.3G	0.30	1.43×	0	11.75
Ours - N=5	156.0G	0.31	1.48×	0	13.68

재학습 비용이 없고 가속화가 잘 됨.

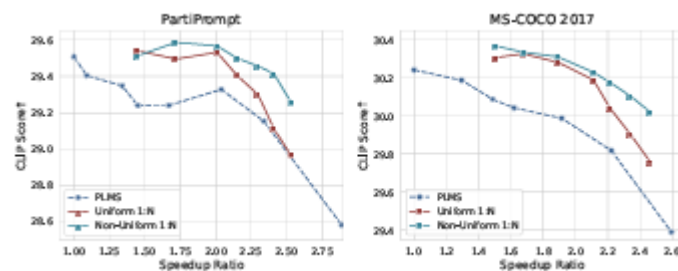
Method	PartiPrompts				COCO2017			
	MACs ↓	Throughput ↑	Speed ↑	CLIP Score ↑	MACs ↓	Throughput ↑	Speed ↑	CLIP Score ↑
PLMS - 50 steps	338.83G	0.230	1.00×	29.51	338.83G	0.237	1.00×	30.30
PLMS - 25 steps	169.42G	0.470	2.04×	29.33	169.42G	0.453	1.91×	29.99
BK-SDM - Base	223.81G	0.343	1.49×	28.88	223.81G	0.344	1.45×	29.47
BK-SDM - Small	217.78G	0.402	1.75×	27.94	217.78G	0.397	1.68×	27.96
BK-SDM - Tiny	205.09G	0.416	1.81×	27.36	205.09G	0.415	1.76×	27.41
Ours	130.45G	0.494	2.15×	29.46	130.45G	0.500	2.11×	30.23

노이즈 제거 속도 빠름



이미지-텍스트 프롬프트가 더 잘 일치함

4.4 Comparison with Fast Sampler



DeepCache는 기존 샘플링 방법(DDIM, PLMS) 대비 속도와 품질에서 비슷하거나 약간 더 나은 성능을 보였으며, 특히 Stable Diffusion과 같은 모델에서 효과를 입증.

5. Limitation

사전 학습된 모델 구조에 의존하며, 얇은 스킵 브랜치가 전체 계산의 큰 비중을 차지하거나 캐싱 단계가 많을수록 성능 저하와 가속 비율의 제약이 발생함

6. Conclusion

DeepCache는 U-Net 구조를 활용해 고차원 특징 계산을 최적화함으로써 추론 속도를 가속화하며, 기존 압축 및 샘플링 방법 대비 유사하거나 더 나은 생성 품질을 제공하는 새로운 접근법임.